

From Five Raw Signals to an Efficiency Grade: A Fog-to-Cloud Monitor for a Two-Array Solar Farm

Mahek Naaz

Student ID: X24217808

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x24217808@student.ncirl.ie

Abstract—A utility solar array is judged not by any single reading but by whether it is turning the sunlight it receives into the power it should. Five signals describe that conversion: the irradiance falling on the panels, the panel temperature, the inverter’s power output, the DC-bus voltage, and the soiling that dulls the glass. They do not play equal roles. Irradiance is the driver the array cannot control, while a hot panel, a dropped output, a sagging voltage, or a dusty surface each points to a fault an operator can act on. Working this out by streaming every raw sample from every string to a distant region wastes bandwidth on values that individually seldom change a decision. This paper presents a monitor that splits the work across a fog tier and a serverless cloud. Ten sensor processes drive five metrics across two arrays; a fog node on the edge host windows each array-and-metric stream, reduces it to a summary, and evaluates the fault rules there, so only finished summaries and the faults they raise cross the network. A serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists those summaries, and a dashboard served through Amazon API Gateway does the one piece of interpretation that needs the whole history: it fuses each array’s output and temperature into a graded efficiency index from zero to one hundred and paints it as a live per-window heatmap, with the static frontend delivered from Amazon S3. Each tier is justified against the alternatives it displaces, and the system was provisioned to a real Amazon Web Services account rather than to a local emulator. An automated suite of 99 tests passes, and verification against the live endpoint confirmed both arrays streaming all five metrics, the efficiency heatmap grading each window, and the fault banner correctly flagging an underperforming array.

Keywords—solar photovoltaic monitoring, renewable energy, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

A solar farm is an array of promises about conversion. Each string is supposed to take the sunlight that lands on it and turn a predictable fraction of that energy into electricity, and the operator’s real question is never the value of one sensor but whether that conversion is happening as it should. Five signals, read together, answer it. Plane-of-array irradiance says how much energy is arriving; inverter output says how much is leaving as power; panel temperature and DC-bus voltage say whether the hardware is in the state that lets the two line up; and a soiling index says whether dust on the glass is quietly stealing yield. The signals also differ in kind, and that difference is the heart of the design: irradiance is an

environmental input the array cannot change and so is never itself a fault, whereas a panel running too hot, an inverter falling short, a voltage sagging below its floor, or a soiling reading past its limit each names a condition someone can act on. Photovoltaic-monitoring practice frames exactly this as the task, turning multi-signal plant telemetry into timely, actionable indicators of underperformance rather than storing raw curves [1], [2].

The distributed-systems question is where that interpretation happens. Sending every sample from every string to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, seldom move a decision, and it defers the one moment that matters, a metric crossing its limit, until after a network round trip. The Internet of Things makes dense sensing cheap [3], but the value is in the interpretation, not the raw stream. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that is idle most of the time [8].

This project builds that two-tier design for a scaled but faithful slice of a solar farm: two arrays, designated array-1 and array-2, each carrying the same five metrics, run as ten independent sensor processes. A fog node on the edge host receives their readings, groups each array-and-metric pair into a fixed time window, reduces the window to summary statistics, evaluates the fault rules against the summary, and forwards only the finished window and any fault it raises. A serverless cloud pipeline persists each window, and a dashboard does the single piece of synthesis that needs more than one signal at once: it fuses each array’s inverter output and panel temperature into a graded efficiency index and renders the recent windows as a colour-banded heatmap, so a supervisor reads array health as a grade rather than as five separate traces.

The work pursues four objectives, each shown on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous

metrics at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a serving layer that turns stored windows into an operational reading, in particular the derived efficiency grade a supervisor could act on without inspecting raw values. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-array scale rather than a full multi-string plant. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one metric on one array. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn uniformly from a small symmetric range and clamps the result into a physically plausible band for that metric, so consecutive readings stay close together like a real instrument trace rather than independent noise. The plausible bands differ sharply by metric, and that difference is the heterogeneity the design has to carry: irradiance ranges across more than a thousand watts per square metre, inverter output across tens to a hundred and fifty kilowatts, DC voltage in the hundreds of volts, and the soiling index as a small percentage. Each process runs two independent timing loops on separate threads, one for sampling and one for dispatch, and each loop waits on a shared shutdown event whose timeout doubles as the tick delay, so a stop signal interrupts a wait immediately instead of after a full interval. The sampling interval defaults to two seconds and the dispatch interval to ten, and keeping them genuinely separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. On each dispatch the process swaps its sample buffer out under a short lock and posts the batch as a compact JSON object naming the metric, the array, and the unit and carrying the handful of timestamped values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte; if that post fails because the fog node is briefly unreachable, the batch is put back so nothing is dropped, and the input interface stays an ordinary HTTP call

the sensors make rather than a shared file or a direct write to storage.

The fog node is a Python service built on an asynchronous web framework, running as its own container on the edge host, distinct from the sensor processes and from the cloud functions. Every ingest request is validated by a separate, event-loop-free routine before anything is buffered, so a malformed batch is rejected with a clear error rather than corrupting a window. Accepted readings are folded into the open window by a combiner buffer whose design is the piece that most repays attention. The buffer holds two dictionaries, an active one that arriving readings extend and a flushing one that is idle, both guarded by a single lock. When the window boundary arrives, the boundary operation swaps the two dictionary references in one step and hands back the groups that had accumulated, so a fresh empty window is in place immediately and the closed window is drained without copying every reading out of it. This makes the rollover an $O(1)$ reference swap rather than a proportional copy, which matters because readings keep arriving from ten sources while the window has to close cleanly on a fixed boundary.

When a window closes, each non-empty array-and-metric group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the fault rules. Those rules are organised as a set of interchangeable strategy objects, one per fault condition, each owning the metric it applies to, the field and limit it tests, and the alert key it raises: panel temperature averaging above sixty-five degrees raises a thermal-derate risk, inverter output averaging below fifty kilowatts raises an underperformance flag, a DC voltage whose window minimum falls below three hundred and fifty volts raises an undervoltage fault, and a soiling index averaging above twenty-five per cent raises a cleaning request. Irradiance deliberately carries no rule, because it is the environmental input the array cannot control and so is never itself a fault; keeping it rule-free is what stops a bright or cloudy sky from being mistaken for a plant problem. Organising the rules this way means a new fault is a new small object rather than a new branch in a growing conditional, and the same rule set is grouped by metric and served verbatim on a thresholds endpoint so the dashboard can disclose the limits without the disclosure ever driving evaluation.

The fog node's output interface is a real managed queue rather than a direct write to storage, and the send is deliberately kept off the window-closing path. Closed summaries are handed to an in-process outbox, and a dedicated publisher thread drains that outbox and ships each ready set of summaries to Amazon SQS in a single batched send of up to the service's per-call limit of ten. Separating the publisher from the window loop means a slow or briefly failing send never delays the next window's close, and batching keeps the number of send operations proportional to the number of distinct metrics closing rather than to the sampling rate. At start-up the publisher resolves the queue's address once, retrying while the queue is still being provisioned, because on

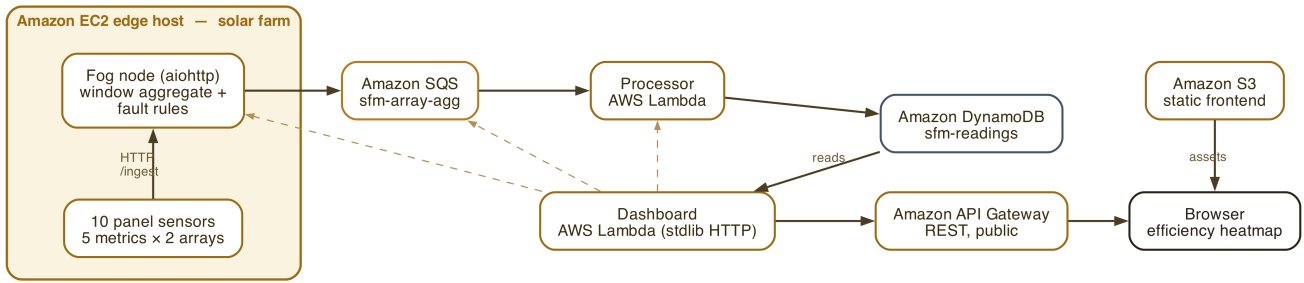


Fig. 1. Deployment topology. Ten sensor processes and the fog node share one Amazon EC2 edge host at the solar farm. Solid arrows are the ingest path, from the fog node through Amazon SQS and the processor function into the Amazon DynamoDB table, and the serving path, from the dashboard function’s read of the stored windows out through Amazon API Gateway to the browser, with static assets from Amazon S3. The dashed edges are the three further status sources the dashboard polls for its health strip: the fog node’s reachability, the queue’s depth, and the processor function’s deployment state.

a fresh deployment the queue may not exist when the edge host boots.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node’s summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure propagates toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer’s behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved up front [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard’s own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then

fetch the most recent windows for a given metric. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its metric as the partition and on a sort value that leads with the window’s closing timestamp and then the array identifier, so windows for one metric are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the array in the sort tail keeps the two arrays’ windows distinct within the same metric.

C. Serving Layer, the Efficiency Grade, and Security

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The read function is written against the Python standard library’s own HTTP handler and is driven inside the function by a small adapter that turns each gateway event into an in-memory request the handler answers exactly as it would a socket, so the same handler code runs unchanged whether it is served locally or invoked as a function and the function carries no web framework at all. It exposes a small, read-only surface: a readings endpoint returning a recent series for one metric, a summary endpoint giving each array’s latest window per metric together with its efficiency history, a thresholds endpoint that forwards the fog node’s published rules, a backend-statistics endpoint, and a health endpoint assembled from four live sources, the store for the freshest window, the queue for its depth, the processor function for its deployment state, and the fog node for its reachability, returning a pipeline flag that is true only when a window actually reached the store recently.

The one piece of interpretation that needs more than a single reading is the efficiency grade, and it is computed here rather than at the edge because it depends on pairing two metrics across a window. For each array the dashboard

pairs the window’s average inverter output with its average panel temperature and maps each onto a zero-to-one-hundred band: output rises from zero at a poor floor to one hundred at an optimal ceiling, temperature falls from one hundred at its optimal ceiling to zero at a hot floor, and the efficiency index is their mean. A rising output band and a falling temperature band capture the physics an operator cares about, that healthy conversion means high output without the array having to run hot, and folding them into one graded number is what lets the page render array health as a colour-banded heatmap, one cell per recent window, instead of asking the viewer to read two traces at once. On the page, that heatmap sits above a card per array showing each metric’s current value against a scaled meter, a cross-array inverter-output trend, and a reference list of the fault rules; a fixed browser-side timer re-fetches the summary, health, and statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node’s health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services’ own access control and on scoping each function to the queue, table, and gateway it needs.

III. IMPLEMENTATION

The sensor processes, the fog node, and both cloud functions are written in Python and run on the Python 3.12 runtime. The fog node is built on an asynchronous web framework and runs its window-flush loop as an event-loop task alongside the dedicated publisher thread; the dashboard reader, by contrast, is a plain standard-library HTTP handler with no framework, driven through the in-memory adapter described above. The AWS SDK for Python carries the queue, table, and function calls, and the frontend renders its trend with a locally vendored charting library so the page has no external dependency at load time. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 99 tests spread across every component: the bounded-walk sensor and its buffer swap, the input validation, the combiner buffer’s active-flushing rollover, the window aggregation arithmetic, the fault rules at and around each limit, the batched publisher, the processor’s transform into a stored item, the dashboard’s data access and health assembly, and the efficiency-index bands and their colour cutoffs. They run without any cloud connection by exercising the units against hand-written fake AWS clients,

so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The fault-rule tests are pointed deliberately at the boundaries, checking a value one step inside a limit and one step outside it, so a rule that should fire and one that should not are both pinned down; the efficiency-index tests check the floor, the ceiling, and a midpoint of each band so the grade cannot drift; and the publisher tests check that a flush is split into batches no larger than the queue’s per-call limit. The suite runs on every push through GitHub Actions, so a change that breaks a rule boundary, an index band, or a query shape fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue’s own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node’s health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both arrays rendered all five metrics with their current value; the efficiency heatmap graded each recent window into its colour band, so a run of good windows and a dip into a lower grade were both visible at a glance; and the fault banner correctly named an underperformance and an undervoltage condition on the array whose output had fallen, while leaving the healthy array unflagged. The backend-statistics endpoint showed the archived record count climbing across successive polls rather than a static number, which confirms the ingestion function was draining the queue and writing on an ongoing basis, and reaching the dashboard from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

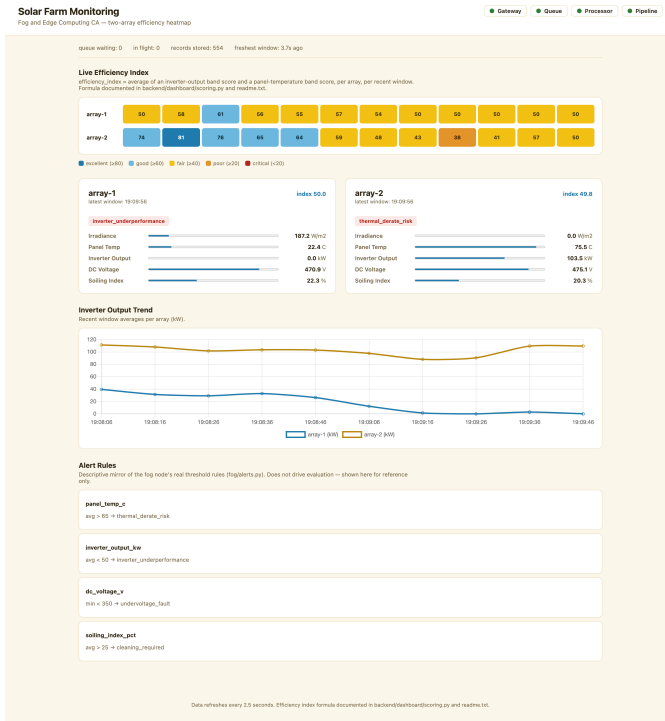


Fig. 2. The deployed dashboard reading from the live endpoint. The efficiency heatmap grades each array’s recent windows into colour bands; each array card shows its five metrics with current value and scaled meters; the cross-array chart shows the inverter-output trend; the header reports arrays tracked, active alerts, and archived records; and the footer strip shows the four pipeline indicators.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five raw solar signals across two arrays into a single live reading of array health, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each array-and-metric stream and evaluates the fault rules locally, so only compact summaries cross the network and a breach is flagged in the window it occurs. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard’s one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 99 automated tests guarding its behaviour and continuous integration running them on every change.

Two reflections stand out from building it. The first is that the value of the two-tier split is clearest where a single reading is not enough: reducing five metrics on very different scales to uniform per-window summaries at the edge is what lets one small record type flow through the queue, the function, and the table, and it is precisely because the summaries are uniform

that the serving layer can afford the one richer step, fusing an array’s output and temperature into a graded efficiency index and painting the recent windows as a heatmap, so raw telemetry becomes a health grade a supervisor reads at a glance. The second is that the design is only honest because it keeps the environmental driver separate from the actionable faults: irradiance is windowed and stored like every other metric but carries no rule, so a bright or overcast sky is never mistaken for a plant fault, and the efficiency grade rewards high output without the array running hot rather than simply rewarding a sunny day.

The work has clear limits. It runs at a two-array scale with simulated sensors, its fault limits and efficiency bands are fixed rather than learned or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to many strings and real inverter feeds, to move the fixed limits to a managed configuration an operator could adjust without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so the efficiency grade can support trend analysis beyond the live view. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-synthesis design carries from two arrays to many without rethinking the tiers.

REFERENCES

- [1] S. R. Madeti and S. N. Singh, “Monitoring System for Photovoltaic Plants: A Review,” *Renewable and Sustainable Energy Reviews*, vol. 67, pp. 1180–1207, 2017.
- [2] A. Woyte, M. Richter, D. Moser, S. Mau, N. Reich, and U. Jahn, “Monitoring of Photovoltaic Systems: Good Practices and Systematic Analysis,” in *Proc. 28th European Photovoltaic Solar Energy Conf.*, 2013, pp. 3686–3694.
- [3] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog Computing and Its Role in the Internet of Things,” in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [5] A. Yousefpour et al., “All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey,” *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [8] M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, “Peeking Behind the Curtains of Serverless Platforms,” in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [11] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [12] M. Elhemali et al., “Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,” in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
- [13] Amazon Web Services, “Amazon Simple Queue Service Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [14] Amazon Web Services, “AWS Lambda Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [15] Amazon Web Services, “Amazon API Gateway Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>

[16] IEEE, “Manuscript Templates for Conference Proceedings,” 2024. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

The source code and deployment configuration are available at [GitHub repository URL].